

COURSE 3 - DATA ENGINEERING 2026

C3-100 - Production Capstone & Engineering Transition

Beginner-first integrated capstone teacher book. RetailPulse grows checkpoint by checkpoint; no lesson is a detached demo.

Artifact	C3-100
Status	2026 - LEARNER EDITION
Phase	C3-09 Production Capstone & Engineering Transition 15 lessons 36-48 focused hours
Prerequisite	C3-08 Gate PASS; all earlier C3 evidence retained.
Assessment boundary	C3-104 is the phase Gate only. C3-113 remains the later course-wide Final Production Gate.
Brand	Charcoal #1F2937 Teal #14B8A6 Soft Blue #3B82F6 AMOLED #000

House teaching rule: requirement -> mental picture -> small proof -> guided integrated build -> expected result -> why it worked -> break/fix -> changed independent case -> validation -> production reasoning -> evidence. Integration is not mastery until the learner can explain and recover the system independently.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

- | | |
|-----------------------------------|---|
| 01. Why it matters | 09. Why it worked |
| 02. What you learn | 10. Common beginner mistakes |
| 03. Prerequisite / starting state | 11. If stuck / recovery |
| 04. Picture it in your head | 12. Small independent task |
| 05. New words before use | 13. Validate |
| 06. Useful video/source | 14. Teach it back |
| 07. Follow with me once | 15. Protected review + changed retry |
| 08. Expected result | 16. Evidence/mastery + exact next route |

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Front matter	Page
This book is your teacher	2
Contents / Index	3
Module map + practice/review/Gate route	4
Phase-specific study notes	5
Lessons	Start
C3-09-L01 Capstone ADR: business problem, architecture and non-functional requirements	6
C3-09-L02 Source contracts, synthetic/public data and reproducible inputs	9
C3-09-L03 Build the batch ingestion path	12
C3-09-L04 Add incremental / CDC-style change handling	15
C3-09-L05 Design layered lakehouse and serving architecture	18
C3-09-L06 Transformation engineering, modeling and tests	21
C3-09-L07 Orchestration, retries, backfills and partial reruns	24
C3-09-L08 Integrate a real Spark component	27
C3-09-L09 Quality gates and source-to-output reconciliation	30
C3-09-L10 Observability, SLOs and operational monitoring	33
C3-09-L11 Security, governance and privacy-safe evidence	36
C3-09-L12 CI/CD, deployment and controlled change	39
C3-09-L13 Incident injection, recovery and prevention	42
C3-09-L14 Runbook, README, handoff and portfolio evidence	45
C3-09-L15 Final architecture defense and engineering transition	48
Mastery checklist + revision quick reference	51

Module map + protected learning route

Production Capstone & Engineering Transition

- 01 Prerequisite / prior-phase Gate
- 02 Lesson mental model + vocabulary
- 03 Follow guided example once
- 04 Matching independent practice
- 05 Save genuine attempt
- 06 Protected review unlocks
- 07 Close review + changed retry
- 08 Validate + explain back
- 09 Master lesson + save evidence
- 10 Mini-Lab / integrated drill
- 11 Protected Gate A
- 12 If needed: protected review -> targeted repair -> changed micro-proof
- 13 Explicitly assign fresh Gate B/C only after repair evidence
- 14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.

How to use this capstone book

One lesson at a time: read the mental model, follow the guided proof, close the book, complete the matching Pxx checkpoint in the RetailPulse starter, save evidence, then open Rxx only after the attempt. Preserve failed attempts; changed retries go into a new evidence folder.

Reuse environments already verified in C3-00 through C3-08. Do not reinstall Airflow, Spark, dbt, Kafka or Fabric tooling merely because C3-09 mentions them. C3-09 is integration: use the appropriate prior-phase environment and record live-runtime status honestly.

Historical C3-09 had zero required videos. RC1 preserves the book-led design while adding optional visuals for learners who benefit from seeing architecture/orchestration/Spark/telemetry/deployment concepts. No Gate depends on external media.

C3-09-L01 - Capstone ADR: business problem, architecture and non-functional requirements

Focused effort: 2.5 h | RetailPulse checkpoint P01 | Review R01 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Start RetailPulse from business users, data products, freshness, recovery, integrity, security, cost and operability - not from a tool checklist.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture a house plan drawn before buying appliances. The ADR states who lives there, what must work, what can fail, and why one layout was chosen. Tools come after the requirements.

5. New words before we use them

ADR = Architecture Decision Record. Functional requirement = what the system must do. NFR = measurable quality requirement such as freshness, recovery, security or cost. Trade-off = benefit accepted with a cost/constraint.

6. Useful video/source

[CURRENT REFERENCE - Azure Well-Architected Framework](#)

Why use it: Architecture/NFR/trade-off framing. Segment: whole relevant page.

[OPTIONAL VISUAL - John Savill - Microsoft Fabric Overview](#)

Why use it: Optional platform architecture orientation. Segment: Use architecture/platform overview; current docs control behavior..

7. Plain-English explanation

An Architecture Decision Record records context, measurable requirements, considered options, decision, consequences and rejected alternatives. A strong capstone can justify a simpler batch-first design when daily freshness is sufficient.

C3-09-L01 - Follow with me once

8. Tiny worked example

RetailPulse needs daily trusted sales by 09:00, one failed day recoverable independently, no secrets in Git, and low idle cost. A batch-first design can satisfy this; Kafka would add complexity without a sub-minute consumer.

9. Follow with me once

Open docs/ADR_001_RETAILPULSE_TODO.md. Write users/data products first, then measurable freshness/recovery/integrity/security/cost NFRs, compare batch-first with one alternative, and record the decision/consequence.

10. Expected result

ADR names users/data products, measurable NFRs, at least two options, chosen design, rejected alternative and consequences. Tool names are secondary to requirements.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If the ADR says only which tools you like, restate the consumer outcome and failure budget before choosing architecture.

13. Common beginner mistakes

- Starting with Kafka/Spark/Fabric names before writing the business outcome.
- Writing vague NFRs such as "fast" or "secure".
- Listing one option only, so no decision was actually made.

C3-09-L01 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Change the requirement from daily to sub-minute fraud detection and state exactly which architectural components must change.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Prefer the simplest architecture that satisfies measurable requirements. Complexity must buy a capability.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P01 first. If repair is needed, open R01, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L01/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save ADR + changed-latency transfer; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L02 - Source contracts, synthetic/public data and reproducible inputs

Focused effort: 2.5 h | RetailPulse checkpoint P02 | Review R02 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Define source grain, stable keys, timestamps, schema, ownership and allowed change while keeping the project reproducible without employer/private data.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture each source as a sealed parcel with a label: one row means what, which field is the identity, when it changed, who owns it, and what changes are allowed. The contract is that label.

5. New words before we use them

Grain = what one row/event represents. Stable key = identity used across changes. Change timestamp/version = ordering signal for updates. Contract = declared schema/semantics/ownership/compatibility. Synthetic data = safe generated data that represents patterns without private records.

6. Useful video/source

[CURRENT REFERENCE - dbt model contracts](#)

Why use it: Contract guarantees and breaking-change thinking. Segment: Why define a contract + breaking changes.

7. Plain-English explanation

A capstone is stronger when another reviewer can rerun it from safe public/synthetic inputs. Versioned source contracts separate source truth from assumptions and make late corrections, schema drift and historical replay explicit.

C3-09-L02 - Follow with me once

8. Tiny worked example

orders has grain one row per order_id; O102 can be corrected using updated_at. Adding a nullable source_note may be backward-compatible; changing amount from numeric to free text is breaking unless migrated.

9. Follow with me once

Open configs/source_contracts_TODO.json. Fill grain/key/change-order/schema/owner for orders and customers. Add one allowed evolution and one breaking-change response.

10. Expected result

Contracts state explicit grains/keys/change semantics/ownership and distinguish compatible versus breaking evolution. No employer/private data is needed.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If you cannot define a stable key or change timestamp, the incremental design is not ready. Record the limitation rather than inventing CDC.

13. Common beginner mistakes

- Using filename as business identity.
- Calling max timestamp "CDC" without source guarantees.
- Using private employer data in a public portfolio.

C3-09-L02 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Add a source field that is optional today but required by a future consumer; define migration and compatibility behavior.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Contracts should survive handoff and platform changes; record uncertainty instead of inventing keys/CDC semantics.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P02 first. If repair is needed, open R02, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L02/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save source contracts + compatibility transfer; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L03 - Build the batch ingestion path

Focused effort: 3.0 h | RetailPulse checkpoint P03 | Review R03 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Land source-preserving evidence with run metadata and prove a repeat of the same batch is safe.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture a loading dock with an untouched receiving area and a separate inspected shelf. Landing preserves what arrived; trusted current state is published only after validation. Replaying the same delivery must not duplicate business state.

5. New words before we use them

Landing = source-preserving intake. Bronze = raw/replay layer. Run ID = identity for one execution. Hash = deterministic fingerprint of bytes. Idempotent = repeating the same logical operation converges to the same business state.

6. Useful video/source

[CURRENT REFERENCE - Python 3.14 csv documentation](#)

Why use it: Current standard-library CSV behavior used by the local proof. Segment: reader/writer/DictReader sections.

7. Plain-English explanation

Batch ingestion should preserve the original source sample, record a deterministic file hash/run identity and separate landing evidence from validated current state. Rerunning an unchanged source must not silently duplicate logical output.

C3-09-L03 - Follow with me once

8. Tiny worked example

orders_2026-09-02.csv is copied to landing with SHA-256 and run_id. Publishing the same file twice leaves one current row per order_id even though two run records may exist.

9. Follow with me once

Create a disposable work folder and run ``python scripts/run_local_reference.py --work work/l03``. Inspect landing files, bronze ingestion metadata, silver current state, gold serving output and evidence summary.

10. Expected result

Local run reports status success, 7 current rows, completed revenue 6910.0 and reconciled=true; source-preserving landing metadata contains hashes/run identity.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If rerun doubles output, separate immutable landing evidence from key-based current-state publication. If source bytes change under the same name, fail or version them explicitly.

13. Common beginner mistakes

- Overwriting source evidence.
- Using append-only publication for current-state data.
- Treating duplicate runs as duplicate business records.

C3-09-L03 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Rerun the same base file twice, then rename it without changing bytes; explain identity behavior.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Keep immutable/source-preserving evidence separate from current-state publication so reruns are auditable and safe.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P03 first. If repair is needed, open R03, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L03/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save batch evidence + rerun proof; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L04 - Add incremental / CDC-style change handling

Focused effort: 3.0 h | RetailPulse checkpoint P04 | Review R04 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Apply corrections/new records deterministically with stable key and update semantics, then prove each logical change occurs once.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture a bookmark plus correction slips. The bookmark tells you how far you processed, while each correction is tied to a stable business key and change time. Advancing the bookmark before safe publish risks losing changes.

5. New words before we use them

Incremental load = process only new/changed input. High-water mark = progress boundary. CDC = change data capture. Upsert/MERGE = insert new keys and update matching keys. Replay = reapply historical input. Tie-break = deterministic rule when versions tie.

6. Useful video/source

[CURRENT REFERENCE - Debezium 3.6 documentation](#)

Why use it: CDC change-event semantics and durable ordering context. Segment: What is it + connector/tutorial concepts.

[OPTIONAL VISUAL - freeCodeCamp - Apache Kafka for Beginners](#)

Why use it: Optional topics/partitions/producers/consumers visual before CDC integration. Segment: Use modern concepts; current Debezium/Kafka docs control exact behavior..

7. Plain-English explanation

Incremental correctness requires more than a watermark. You need a stable key, authoritative change timestamp/version rule, deterministic conflict policy and reconciliation. Late corrections can invalidate simplistic max-timestamp polling assumptions.

C3-09-L04 - Follow with me once

8. Tiny worked example

Base has O102=1000. Incremental has O102=1050 plus O106/O107. After replay, current state still has 7 unique orders and O102=1050 - not 8/9 rows.

9. Follow with me once

Run the local reference with incremental input, inspect O102/O106/O107, rerun to a fresh work folder, and compare current keys/revenue. Then complete scripts/04_incremental_TODO.py.

10. Expected result

O102 is 1050.00, O106 and O107 each appear once, current row count is 7, completed revenue remains 6910.0 after replay.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If an incremental replay changes totals again, your update is not idempotent. If two records tie on change time, declare a deterministic tie-break rule.

13. Common beginner mistakes

- Advancing watermark before validated publish.
- Ignoring late/out-of-order corrections.
- Using nondeterministic tie handling.

C3-09-L04 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Apply a late correction whose updated_at predates the last observed watermark and explain how your source contract prevents or handles loss.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Advance incremental state only after successful validated publish; design for late correction and deterministic ties.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P04 first. If repair is needed, open R04, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L04/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save incremental output + replay/late-correction proof; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L05 - Design layered lakehouse and serving architecture

Focused effort: 3.0 h | RetailPulse checkpoint P05 | Review R05 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Separate source-preserving Bronze, validated/current-state Silver and consumer-serving SQL/Gold contracts without creating layers for presentation.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture three rooms with different trust promises. Bronze keeps replay evidence, Silver holds validated/current business state, and Gold/SQL serves a specific consumer. Layers exist because responsibilities differ, not because three folders look professional.

5. New words before we use them

Lakehouse = data platform combining files/tables and analytics engines. Delta = transaction-log table format. Serving layer = consumer-facing contract. Warehouse = SQL-first serving system. Trust boundary = point where validation/ownership changes.

6. Useful video/source

[CURRENT REFERENCE - Delta Lake 4.4.0 release context](#)

Why use it: Current Spark 4.2/Delta 4.4 lakehouse compatibility and transaction context. Segment: Delta-Spark now defaults to Spark 4.2.

7. Plain-English explanation

Layering is about trust and responsibilities. Bronze preserves replay evidence, Silver enforces contracts/current state, and serving tables/views answer a consumer need. Lakehouse versus Warehouse is a serving/workload decision; Delta provides transactional table behavior where used.

C3-09-L05 - Follow with me once

8. Tiny worked example

Bronze keeps both base and incremental source evidence. Silver contains 7 current orders. Gold contains daily completed revenue. A Warehouse view is added only if SQL consumers need that serving contract.

9. Follow with me once

Map local landing/bronze/silver/gold outputs to your target lakehouse/warehouse design in `architecture/L05_layered_architecture_TODO.md`. State owner, grain, trust rule and retention for each layer.

10. Expected result

Every layer has a different responsibility/trust contract; serving choice is tied to a real consumer pattern; no layer exists only for naming convention.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If every layer contains the same data, explain what trust/transformation boundary is missing. If Warehouse exists only because it sounds enterprise, remove it or justify the SQL consumer.

13. Common beginner mistakes

- Creating Bronze/Silver/Gold copies with no distinct contract.
- Choosing Warehouse only because it sounds enterprise.
- Mixing raw replay retention with curated serving access.

C3-09-L05 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Redesign for a second consumer that needs raw event history instead of current-state orders.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Layer ownership, retention, access and trust are production decisions. Add Warehouse/Eventhouse only for a justified consumer/query pattern.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P05 first. If repair is needed, open R05, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L05/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save layer contracts + alternate-consumer transfer; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L06 - Transformation engineering, modeling and tests

Focused effort: 3.0 h | RetailPulse checkpoint P06 | Review R06 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Create maintainable transformations whose grain, joins, business rules and tests can be independently reproduced.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture a factory line where every station has a measurable input/output grain. Transformation code changes the product; tests and independent controls inspect it. A green run is not enough if the inspection copies the same mistake.

5. New words before we use them

Transformation grain = row meaning at each stage. Referential integrity = relationship keys resolve correctly. Data test = query/assertion over built data. Model contract = declared output shape. Regression = previously correct behavior breaks after change.

6. Useful video/source

[CURRENT REFERENCE - dbt data tests](#)

Why use it: Content validation after model build and test severity concepts. Segment: data tests + relationship to model contracts.

7. Plain-English explanation

Transformation code is production code. Tests should cover grain/keys, accepted domains, referential integrity, critical calculations and changed-input behavior. dbt is one implementation path, not the definition of correctness.

C3-09-L06 - Follow with me once

8. Tiny worked example

A transform groups current Completed orders by order_date. A test checks unique order_id and accepted status; an independent control separately sums source-approved Completed amounts to 6910.0.

9. Follow with me once

Run ``python -m pytest -q``, inspect `tests/test_capstone_core.py`, then complete `transforms/L06_TRANSFORMATION_AND_DBT_TODO.md` with model grains, tests/contracts and one independent reconciliation.

10. Expected result

Tests run from a clean project with ``python -m pytest -q``; current-state grain and daily sales are independently validated; changed-rule impact is bounded and explained.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If a test merely asserts the same logic as the transform, add an independent invariant or manual control. If totals change after a refactor, fail closed.

13. Common beginner mistakes

- Testing only that code executes.
- Writing a test that copies the same transformation logic.
- Hiding duplicate/grain problems with DISTINCT.

C3-09-L06 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Change one business rule for completed revenue and prove the expected result plus regression boundary.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Transformation tests protect shape/content, but independent reconciliation protects against shared logic errors.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P06 first. If repair is needed, open R06, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L06/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save transform + independent tests + changed rule; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L07 - Orchestration, retries, backfills and partial reruns

Focused effort: 3.5 h | RetailPulse checkpoint P07 | Review R07 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Coordinate the capstone by data interval, classify transient versus deterministic failures and prove bounded historical reprocessing.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture a train timetable. Airflow coordinates when each carriage moves, but each task must remain safe if the scheduler retries or replays one interval. Backfill is a bounded historical service, not "rerun everything."

5. New words before we use them

DAG = directed acyclic graph of tasks. Data interval = logical time window a run owns. Retry = repeat after eligible failure. Backfill = intentional historical processing. Partial rerun = rerun only affected work. Failure class = transient vs deterministic/permanent.

6. Useful video/source

[CURRENT REFERENCE - Airflow 3.3.1 architecture overview](#)

Why use it: Current DAG/task/run architecture. Segment: DAG/tasks/components.

[OPTIONAL VISUAL - Astronomer - Building ETL/ELT pipelines with Airflow 3](#)

Why use it: Optional end-to-end Airflow 3 workflow/reliability visual. Segment: Focus on orchestration, quality, backfills; page exposes no verified chapters..

7. Plain-English explanation

Orchestration should make dependencies, interval identity, retries, backfills and partial reruns explicit. A retry is safe only when the underlying task is idempotent. Historical reprocessing must target a bounded interval without corrupting other dates.

C3-09-L07 - Follow with me once

8. Tiny worked example

A source read timeout may retry; a schema-contract failure should stop. Reprocessing only 2026-09-02 must not mutate 2026-09-01 or 2026-09-03 outputs.

9. Follow with me once

Complete orchestration/L07_AIRFLOW_DAG_TODO.py using Airflow 3 concepts from C3-03. State interval identity, retry classes, dependencies, backfill range and partial-rerun safety. Execute live DagRun only in your verified Airflow environment.

10. Expected result

DAG/design distinguishes retryable vs deterministic failure, scopes work by data interval, supports one bounded backfill and proves partial rerun does not duplicate/corrupt state.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If every failure retries, classify deterministic data defects separately. If backfill overwrites all history, restore partition/date-scoped publication.

13. Common beginner mistakes

- Retrying deterministic data defects.
- Backfilling all dates to repair one partition.
- Clearing tasks without understanding side effects.

C3-09-L07 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Backfill one historical service date after changing only its source file and prove unrelated dates are unchanged.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Scheduler retries do not make unsafe tasks idempotent; historical reprocessing needs bounded scope and post-run reconciliation.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P07 first. If repair is needed, open R07, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L07/retry_001; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save DAG/design + backfill evidence; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L08 - Integrate a real Spark component

Focused effort: 3.5 h | RetailPulse checkpoint P08 | Review R08 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Use Spark where distributed processing changes the execution problem, and keep a parity contract against a deterministic small-data reference.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture moving from a kitchen scale to a warehouse conveyor. Spark is useful when data must be partitioned and processed across workers. On tiny capstone data, its purpose is proving distributed execution reasoning and parity with a small reference.

5. New words before we use them

Driver = coordinates Spark work. Executor = performs distributed tasks. Partition = chunk of distributed data. Shuffle = data redistribution across workers. Parity invariant = same business result between Spark and trusted reference on the same fixture.

6. Useful video/source

[CURRENT REFERENCE - Spark 4.2.0 documentation](#)

Why use it: Current Spark/Python/Java compatibility and distributed execution. Segment: overview + quick start.

[OPTIONAL VISUAL - Spark architecture visual](#)

Why use it: Optional driver/executor/partition visual. Segment: Focus driver/workers/partitions/jobs/stages/tasks/shuffle..

7. Plain-English explanation

Spark belongs in the capstone because the course must prove distributed transformation reasoning, not because small synthetic data requires a cluster. Keep an explicit input/output contract and compare Spark output to the local reference on the same fixture.

C3-09-L08 - Follow with me once

8. Tiny worked example

Spark reads the same 7 current orders and computes the same customer/order aggregate as the local Python reference. If the values differ, correctness is fixed before partition tuning.

9. Follow with me once

Complete `spark/L08_SPARK_JOB_TODO.py` using the C3-05 Spark environment. Run the same fixture through Spark and save parity with the deterministic reference before any optimization claim.

10. Expected result

Spark output matches the local reference on agreed keys/counts/values; scale note explains why Spark is not the cheapest choice for the tiny fixture but what changes at larger scale.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If Spark output disagrees with the reference, fix correctness before tuning. If the dataset is tiny, state that Spark is scale-transfer evidence rather than a cost-optimal local choice.

13. Common beginner mistakes

- Using Spark on tiny data and calling it "big data".
- Tuning before parity is correct.
- Claiming scale numbers without measured evidence.

C3-09-L08 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Explain the first bottlenecks/partition changes at 10x and 100x volume without inventing a cluster benchmark.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Treat Spark as distributed execution, not decoration. Correctness/parity comes before caching, partitioning or performance claims.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P08 first. If repair is needed, open R08, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L08/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save Spark component + parity/scale note; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L09 - Quality gates and source-to-output reconciliation

Focused effort: 3.0 h | RetailPulse checkpoint P09 | Review R09 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Block unsafe publication and reconcile source, current state and serving outputs with keys, counts, values and business totals.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture airport security gates. Data can move through the pipeline only if required checks pass. Reconciliation is the independent passenger manifest that proves the people who arrived are the people expected - not merely the same headcount.

5. New words before we use them

Quality gate = check that can block unsafe publish. Reconciliation = independent comparison of source/target keys/counts/values/totals. Invariant = condition that must always hold. Severity = operational consequence of a failed assertion.

6. Useful video/source

[CURRENT REFERENCE - OpenLineage Data Quality Assertions](#)

Why use it: Assertion result/severity and dataset-linked quality evidence. Segment: success vs severity + assertion fields.

7. Plain-English explanation

A capstone is not trustworthy because every component ran. Quality gates protect declared invariants; reconciliation proves that the published data corresponds to accepted source changes. Use independent controls before and after backfills/recovery.

C3-09-L09 - Follow with me once

8. Tiny worked example

A corrupt O104 amount reduces completed revenue by 100. Row count remains 7, so a count-only gate misses it; key/value and business-total reconciliation catches it.

9. Follow with me once

Run ``python scripts/reference_controls.py``. Then complete `quality/L09_QUALITY_RECONCILIATION_TODO.md` and demonstrate a failing control on a disposable corrupted copy.

10. Expected result

Quality gate blocks injected corruption even when row count alone could look correct; post-repair keys/values/totals reconcile exactly.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If row counts match but values differ, compare key sets and record-level values. If totals match but keys differ, treat total parity as insufficient.

13. Common beginner mistakes

- Checking only row count.
- Using the same derived result as its own reconciliation.
- Allowing warning-only checks for critical integrity invariants.

C3-09-L09 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Create an offsetting defect that preserves the grand total and prove the key/value reconciliation still catches it.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Use multiple independent controls: keys + counts + values/totals + business invariants. One passing total is never sufficient.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P09 first. If repair is needed, open R09, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L09/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save quality gate + reconciliation evidence; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L10 - Observability, SLOs and operational monitoring

Focused effort: 3.0 h | RetailPulse checkpoint P10 | Review R10 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Emit correlated run evidence, define meaningful SLIs/SLOs and route alerts to an owner/runbook.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture a control room with a shared run_id on every signal. Logs tell events, metrics show trends, and traces/correlation connect work. SLOs translate those signals into a promise a consumer actually cares about.

5. New words before we use them

Telemetry = operational signals. Log = discrete event record. Metric = numeric time series. Trace/correlation = link across operations. SLI = measured reliability signal. SLO = target for that signal. Alert = condition that triggers action.

6. Useful video/source

[CURRENT REFERENCE - OpenTelemetry Overview](#)

Why use it: Logs/metrics/traces/resources/context model. Segment: signals + resource/context sections.

[OPTIONAL VISUAL - Logs, Metrics, Traces: Correlate with OpenTelemetry](#)

Why use it: Optional correlated debugging visual. Segment: 0:58 logs->traces; 2:39 logs->metrics; 3:47 semantic mapping..

7. Plain-English explanation

Logs answer discrete events, metrics quantify behavior over time and traces/correlation connect components. For a data product, useful SLIs include validated publish success, freshness, completeness and duration. Alerts must map to user impact and a response path.

C3-09-L10 - Follow with me once

8. Tiny worked example

run_id=rp-009 links ingest_complete, publish_complete and reconcile_complete. Freshness SLI is publish_time-source_cutoff; validated-publish SLI requires reconciliation success.

9. Follow with me once

Open work evidence/run_summary.json and events.jsonl after a local run. Complete observability/L10_SLO_ALERT_TODO.md with at least two SLIs, targets, alert routing and owner/runbook.

10. Expected result

SLIs/SLOs are measurable, run evidence is correlated, alerts have owner/action/runbook, and page-worthy versus ticket-worthy conditions are justified.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If an alert has no owner/runbook or fires on every harmless anomaly, redesign it around consumer impact and actionable evidence.

13. Common beginner mistakes

- Alerting on every anomaly.
- No run_id/correlation across logs.
- SLO defined from an internal metric with no consumer impact.

C3-09-L10 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Change the business cutoff/SLO window and explain the alert effect without changing pipeline code.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Operational signals exist to support decisions. Every alert needs a user-impact rationale, owner and recovery path.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P10 first. If repair is needed, open R10, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L10/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save SLO/alert policy + correlated run evidence; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L11 - Security, governance and privacy-safe evidence

Focused effort: 3.0 h | RetailPulse checkpoint P11 | Review R11 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Apply least privilege across repository/workflow/data access and keep secrets/PII out of portfolio evidence.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture two locks: one on the control panel and one on the data cabinet. Repository/workflow permissions protect changes; data-plane permissions protect records. Portfolio evidence is a photocopy with secrets and private identifiers removed.

5. New words before we use them

Least privilege = minimum permissions required. Control plane = permission to manage resources/workflows. Data plane = permission to read/write data. Secret = credential/token/key. OIDC = federated identity exchange for short-lived cloud access. Sanitization = remove/redact sensitive values from evidence.

6. Useful video/source

[CURRENT REFERENCE - OneLake security overview](#)

Why use it: Current control-plane vs data-plane and role behavior. Segment: control plane + data plane + role effects.

7. Plain-English explanation

Security evidence must show both allow and deny boundaries where possible. Long-lived cloud credentials should be avoided when federated/OIDC identity is supported. Portfolio screenshots/logs should be synthetic/sanitized and never expose tokens, account keys or private tenant identifiers.

C3-09-L11 - Follow with me once

8. Tiny worked example

A workflow may read repository code but should not receive production write access before deployment approval. A portfolio screenshot replaces tenant IDs/tokens/private names with sanitized placeholders.

9. Follow with me once

Run ``python scripts/static_project_audit.py``. Complete security/L11_ACCESS_GVERNANCE_TODO.md with repository/workflow/data access boundaries and one intended-consumer deny case.

10. Expected result

Static secret scan is clean; permissions are least-privilege by role; one deny boundary is specified/proven where runtime permits; portfolio evidence contains no private secrets/PII.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If a workflow needs broad write/admin access for convenience, reduce scope. If evidence contains real secrets/PII, rotate/revoke if needed and rebuild sanitized evidence.

13. Common beginner mistakes

- Putting secrets in screenshots/logs.
- Testing data restrictions as an admin.
- Granting broad roles just to make a demo work.

C3-09-L11 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Add a contractor who can read curated output but must not read raw/landing data; show the control-plane/data-plane distinction.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Use least privilege, short-lived identity when supported, consumer-identity tests and sanitized evidence. Security screenshots are not proof by themselves.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P11 first. If repair is needed, open R11, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L11/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save access matrix + deny case + sanitized evidence; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L12 - CI/CD, deployment and controlled change

Focused effort: 3.0 h | RetailPulse checkpoint P12 | Review R12 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Make pull requests fail closed on tests/quality and promote a reviewed artifact through protected environments with post-deploy validation.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture a guarded bridge from source control to production. CI tests the artifact before entry; environment protection controls who may cross; OIDC can provide a short-lived credential at crossing time; post-deploy checks prove the data is still correct.

5. New words before we use them

CI = automated build/test on change. CD = controlled promotion/deployment. Environment = deployment target such as dev/test/prod. Protection rule = approval/branch/custom gate before deploy. Rollback = return to earlier version. Roll-forward = fix with a new version.

6. Useful video/source

[CURRENT REFERENCE - GitHub deployments and environments](#)

Why use it: Protection rules, environment secrets and deployment boundaries. Segment: deployment protection rules + environment secrets.

[CURRENT REFERENCE - GitHub Actions OIDC in cloud providers](#)

Why use it: Short-lived federated cloud identity. Segment: permissions + provider token exchange.

[OPTIONAL VISUAL - Continuous delivery with GitHub Actions](#)

Why use it: Optional protected-environment visual. Segment: 3:55 protect environments; 5:36 scoped secrets; 7:02 deployments; 8:44 demo..

7. Plain-English explanation

CI proves a clean runner can install dependencies and run deterministic checks. CD separates immutable artifact from environment configuration, uses approvals/branch restrictions where appropriate, and validates data after deployment. OIDC can replace stored long-lived cloud secrets.

C3-09-L12 - Follow with me once

8. Tiny worked example

Pull request -> install -> tests -> static checks -> protected production environment -> short-lived cloud identity -> deploy -> reconciliation smoke. If tests fail, deploy never runs.

9. Follow with me once

Inspect `.github/workflows/ci_TODO.yml` and complete `cicd/L12_RELEASE_PIPELINE_TODO.md`. Keep tests upstream of deploy, use protected environment reasoning, minimum permissions, OIDC where supported, and post-deploy data validation.

10. Expected result

CI fails closed on tests/checks; deploy is protected; credentials are not unnecessarily long-lived; post-deploy smoke includes data reconciliation, not only workflow success.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If deploy can run when tests fail, restore dependency. If prod credentials are available before approval, fix the environment boundary.

13. Common beginner mistakes

- Deploy job can run after failed tests.
- Production secrets accessible before approval.
- Calling successful deployment proof of data correctness.

C3-09-L12 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Introduce a schema/business-rule change and specify rollback versus roll-forward plus post-change reconciliation.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Promote immutable/reviewed artifacts through protected environments; separate configuration/secrets; validate data after deployment.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P12 first. If repair is needed, open R12, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L12/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save CI design + release checklist + safe-change evidence; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L13 - Incident injection, recovery and prevention

Focused effort: 3.5 h | RetailPulse checkpoint P13 | Review R13 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Inject a controlled defect into disposable evidence, diagnose it, repair the smallest scope, reprocess safely and prove prevention.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture a fire drill using a disposable training room. You inject one controlled defect, contain it, diagnose it from evidence, repair the smallest scope, reprocess safely, and add prevention. You never burn the real building to prove the alarm works.

5. New words before we use them

Incident = unexpected service/data failure requiring response. Containment = limit blast radius. Root cause = underlying reason. Recovery = restore safe service/data. Post-recovery validation = prove trust after repair. Prevention = test/control/runbook change reducing recurrence.

6. Useful video/source

[CURRENT REFERENCE - Google SRE Workbook - Incident Response](#)

Why use it: Containment, coordination, working record and recovery. Segment: incident response chapter.

7. Plain-English explanation

Production credibility comes from demonstrated failure handling. Never damage canonical data just to create a demo. Preserve incident timeline, evidence, containment, root cause, repair, reprocess/backfill, post-recovery reconciliation and preventive tests/runbook changes.

C3-09-L13 - Follow with me once

8. Tiny worked example

Inject O104 amount corruption in a copy. Gate blocks publish, evidence points to O104, repair uses source truth, bounded rerun restores 6910.0, and a regression test prevents recurrence.

9. Follow with me once

Run ``python scripts/incident_drill_reference.py``. Complete `incident/L13_INCIDENT_REPORT_TODO.md` with timeline, containment, diagnosis, bounded repair, revalidation and one prevention change.

10. Expected result

Incident evidence shows defect detection -> containment -> root cause -> bounded repair/replay -> post-recovery reconciliation -> prevention update.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If recovery deletes/rebuilds everything without need, reduce blast radius. If you cannot prove the repaired result, the incident remains open.

13. Common beginner mistakes

- Damaging canonical evidence to create a demo.
- Deleting/rebuilding everything instead of bounded recovery.
- Closing incident before post-recovery reconciliation.

C3-09-L13 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Repeat with a different defect class such as missing customer FK or duplicate order key.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Minimize blast radius. Preserve evidence. Rebuild only what the incident requires and prove trust before closing it.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P13 first. If repair is needed, open R13, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L13/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save incident report + bounded recovery + prevention; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L14 - Runbook, README, handoff and portfolio evidence

Focused effort: 2.5 h | RetailPulse checkpoint P14 | Review R14 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Make the repository reproducible and interview-ready without turning documentation into marketing claims.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture handing your project to another engineer while you leave the room. README helps them understand/build it; runbook helps them operate/recover it; the evidence index points to proof. If they need your memory, the handoff is incomplete.

5. New words before we use them

README = project orientation/setup/use. Runbook = operational detect/diagnose/recover/validate steps. Evidence index = map from claim to proof. Handoff = another engineer can reproduce and operate the project. Limitation = explicit capability not proven.

6. Useful video/source

[CURRENT REFERENCE - GitHub - About README files](#)

Why use it: Repository orientation and reproducibility expectations. Segment: what a README should communicate.

7. Plain-English explanation

README explains architecture/setup/run/test/evidence and known limits. Runbook explains operating preconditions, commands, success signals, failure diagnosis, safe reruns/backfills and escalation. Portfolio evidence should show decisions, tests, recovery and trade-offs - not just screenshots.

C3-09-L14 - Follow with me once

8. Tiny worked example

A fresh reviewer follows README to create the venv, runs tests, generates local evidence, then uses runbook to diagnose a simulated failure without asking the author for hidden steps.

9. Follow with me once

Complete docs/README_TODO.md, runbooks/production_runbook_TODO.md, docs/portfolio_evidence_index_TODO.md and docs/L14_HANDOFF_PORTFOLIO_TODO.md from evidence you actually generated.

10. Expected result

A fresh reviewer can set up, run, test, troubleshoot and understand evidence using only README/runbook; claims are linked to sanitized proof and limitations are explicit.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If another engineer needs your memory to run the project, document hidden prerequisites. If a claim is not backed by evidence, weaken or remove it.

13. Common beginner mistakes

- README is marketing only.
- Runbook lacks exact success/failure signals.
- Portfolio claims live tools that were never executed.

C3-09-L14 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Hand the repository to a fresh reviewer using only README/runbook and record every missing instruction.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Documentation is part of operability. Another engineer should reproduce the system without undocumented local knowledge.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P14 first. If repair is needed, open R14, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L14/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save README + runbook + evidence index + handoff gap log; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

C3-09-L15 - Final architecture defense and engineering transition

Focused effort: 3.0 h | RetailPulse checkpoint P15 | Review R15 locked until attempt saved

1. Why it matters

A capstone proves integration and ownership. A disconnected demo or copied snippet is weaker than a smaller coherent system with explicit requirements, failure boundaries and independent validation.

2. What you learn

Defend RetailPulse independently, identify 10x-scale changes and map concrete evidence to junior data-engineering interviews without claiming unearned production experience.

3. Prerequisite

C3-08 Gate PASS and retained evidence from C3-00 through C3-08. Use the prior verified tool environment for any live Airflow/Spark/dbt/Fabric/Kafka work; new concepts in this lesson are still explained here.

4. Picture it in your head

Picture an architecture interview as defending a map, not reciting tool definitions. For every component you should answer: why here, what data enters/leaves, how it fails, how you detect/recover it, and what changes at 10x scale.

5. New words before we use them

Architecture defense = explain design choices under questioning. Bottleneck = resource/step limiting throughput or reliability. Scale transfer = how design changes with larger load. Vendor-specific = tied to one platform. Permanent principle = general engineering rule that survives tool change.

6. Useful video/source

[CURRENT REFERENCE - DP-700 Study Guide](#)

Why use it: Optional certification objective map after engineering evidence exists. Segment: skills measured as of July 21, 2026.

7. Plain-English explanation

Architecture defense is an oral/system reasoning test: why each component exists, what happens when it fails, how correctness is proved, which parts are vendor-specific and what changes at larger scale. C3-104 is this phase Gate; C3-113 remains the later course-wide Final Production Gate.

C3-09-L15 - Follow with me once

8. Tiny worked example

Asked "Why Spark?" you answer: small fixture does not need it; Spark is included to prove distributed parity/scale reasoning. At 100x volume you would inspect partition sizes, shuffle and runtime evidence before tuning.

9. Follow with me once

Use docs/architecture_defense_questions.md. Record a 10-15 minute defense without notes, list gaps, then complete docs/L15_ARCHITECTURE_DEFENSE_TODO.md. C3-104 comes only after all prior C3-09 evidence is ready.

10. Expected result

Defense explains every major component, failure/recovery/evidence path, vendor-specific boundary and 10x/100x change. C3-104 is understood as phase Gate, not graduation.

11. Why it worked

The checkpoint has an explicit requirement, input/output grain or resource boundary, a deterministic success condition and a separate validation path. The result is accepted because the evidence agrees with the contract - not merely because a command returned exit code 0.

12. Break / fix / debug

If you answer with tool definitions instead of system behavior, return to requirements, data flow, failure mode and evidence. Do not claim live cloud/Spark/Airflow execution you did not perform.

13. Common beginner mistakes

- Answering with definitions instead of data flow/failure reasoning.
- Claiming benchmark/production experience not evidenced.
- Confusing C3-104 phase pass with C3-113 graduation.

C3-09-L15 - Do alone, validate and master

14. Recovery if stuck

1) Restate the requirement and current input/output grain. 2) Identify the first missing/incorrect evidence. 3) Reproduce the smallest failing step. 4) Use the current source above only for syntax/current behavior. 5) Reveal only the next structural hint. 6) Open Rxx only after the genuine attempt is saved.

15. Independent small version

Defend a variant where Fabric is replaced by another lakehouse/cloud platform and separate permanent engineering principles from vendor features.

16. Validation

Use an independent validation appropriate to the checkpoint. Never count the same calculation as both transformation and proof. Preserve prior passing evidence when testing a changed failure or assumption.

17. Production / architecture reasoning

Interview readiness means explaining system behavior and trade-offs from evidence without overstating production experience.

18. Teach it back

Explain in 60-90 seconds why this component exists, how it fails, how you detect/recover it, what evidence proves correctness and what changes at larger scale.

19. Protected review + changed retry

Save P15 first. If repair is needed, open R15, compare only the weak concept, close it, then solve a changed case from the starter/evidence workspace. Write the retry to evidence/L15/retry_001/; never overwrite attempt_001.

20. Evidence / mastery + exact next route

Save defense notes + evidence map + readiness gaps; include confidence 1-5, main uncertainty, independent validation and one honest runtime/platform limitation. PASS requires the artifact plus observed result/runtime status, an independent control, one failure/recovery note and a concise own-words defense. PASS -> next lesson; PARTIAL/FAIL -> repair only this competency and retry on changed data.

Mastery checklist + revision quick reference

- ☐ Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- ☐ Protected review opened only after the attempt; changed retry solved after closing the review.
- ☐ Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- ☐ Mini-Lab / integrated drill completed with reproducible evidence.
- ☐ Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- ☐ No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- ☐ Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.